Pertemuan 9-10: Menambah Kolom, Mengurutkan, Menggabungkan Data

STA623211 - Basis Data dan SQL

Structured Query Language (**SQL**) adalah bahasa yang digunakan untuk mengelola dan mengakses data dalam sistem manajemen basis data relasional. Modul ini berfokus pada tiga operasi SQL dasar:

1. Menambah kolom pada Tabel
2. Mengurutkan data
3. Menggabungkan Tabel

Membuat Tabel Dasar

Membuat Tabel `mahasiswa` dan `matkul`:

```
CREATE DATABASE IF NOT EXISTS db1;
USE db1;

CREATE TABLE mahasiswa (
    id_mhs INT AUTO_INCREMENT PRIMARY KEY,
    nama VARCHAR(100),
    jurusan VARCHAR(100)
);

CREATE TABLE mk (
    kode_mk INT AUTO_INCREMENT PRIMARY KEY,
    nama_mk VARCHAR(100)
);
```

Melihat tabel yang sudah dibuat:

```
SHOW TABLES;
```

```
##   Tables_in_db1
## 1     mahasiswa
## 2            mk
```

`AUTO_INCREMENT` adalah atribut yang bisa diterapkan pada kolom berjenis integer (biasanya sebagai `PRIMARY KEY`) agar **nilainya secara otomatis bertambah** satu setiap kali data baru dimasukkan ke dalam tabel. Biasanya digunakan pada kolom `id_mhs` atau `primary key` untuk memberikan nilai unik pada setiap baris, tanpa harus memasukkan nilainya secara manual.

Memasukkan data ke dalam tabel

Masukkan beberapa data ke dalam tabel `mahasiswa`:

```
INSERT INTO mahasiswa
VALUES
(NULL, 'Agus', 'Manajemen'),
(NULL, 'Budi', 'Akuntansi'),
(NULL, 'Chintia', 'Informatika'),
(NULL, 'Dewi', 'Akuntansi')
;
```

Masukkan beberapa data ke dalam tabel `mk`:

```
INSERT INTO mk
VALUES
(NULL, 'Matematika Dasar'),
(NULL, 'Pengantar Data Sains'),
(NULL, 'Agama')
;
```

CATATAN: Jika ingin mengisi semua kolom tanpa menyebut nama kolom secara eksplisit, urutan nilai dalam `VALUES` harus sesuai dengan urutan kolom dalam tabel. Di sini, NULL digunakan untuk kolom id_mhs yang diatur AUTO_INCREMENT.

Selanjutnya, kita dapat melihat tabel `mahasiswa` dan tabel `mk` yang sudah ditambahkan beberapa data tersebut.

```
SELECT * FROM mahasiswa;
SELECT * FROM mk;
```

```
##   id_mhs    nama     jurusan
## 1      1    Agus   Manajemen
## 2      2    Budi   Akuntansi
## 3      3 Chintia Informatika
## 4      4    Dewi   Akuntansi
```

```
##   kode_mk              nama_mk
## 1       1     Matematika Dasar
## 2       2 Pengantar Data Sains
## 3       3                Agama
```

1. Menambah Kolom pada Tabel

Untuk menambah kolom baru pada tabel yang sudah ada dalam basis data, kita menggunakan perintah `ALTER TABLE` di SQL. Struktur perintah SQL untuk menambahkan kolom baru adalah sebagai berikut.

```
ALTER TABLE nama_tabel
ADD COLUMN nama_kolom TIPE_DATA;
```

- **nama_tabel**: Nama tabel yang ingin diubah.

- `ADD COLUMN`: Perintah untuk menambahkan kolom baru ke tabel.
- **nama_kolom**: Nama kolom baru yang ingin ditambahkan.
- **TIPE_DATA**: Tipe data untuk kolom baru (misalnya, INT, VARCHAR, DATE).

Penambahan beberapa kolom juga dapat dilakukan sekaligus dalam satu perintah.

```
ALTER TABLE nama_tabel
ADD COLUMN nama_kolom_1 TIPE_DATA_1,
ADD COLUMN nama_kolom_2 TIPE_DATA_2,
ADD COLUMN nama_kolom_3 TIPE_DATA_3;
```

Sebagai contoh, kita akan menambahkan kolom `tanggal_lahir` pada tabel `mahasiswa`, dan `semester` pada tabel `mk`.

```
ALTER TABLE mahasiswa
ADD COLUMN tanggal_lahir DATE;

ALTER TABLE mk
ADD COLUMN semester INT;
```

Tampilkan struktur tabel `mahasiswa` untuk memeriksa bahwa kolom baru telah ditambahkan.

```
DESCRIBE mahasiswa;
DESCRIBE mk;
```

```
##           Field         Type Null Key Default          Extra
## 1        id_mhs          int   NO PRI    <NA> auto_increment
## 2          nama varchar(100)  YES        <NA>
## 3       jurusan varchar(100)  YES        <NA>
## 4 tanggal_lahir         date  YES        <NA>

##      Field         Type Null Key Default          Extra
## 1  kode_mk          int   NO PRI    <NA> auto_increment
## 2  nama_mk varchar(100)  YES        <NA>
## 3 semester          int  YES        <NA>
```

1.1. Memasukkan data ke kolom baru berdasarkan ID

Setelah ditambahkan kolom `tanggal lahir` dan `semester`, tabel `mahasiswa` dan `mk` menjadi seperti berikut.

```
SELECT * FROM mahasiswa;
SELECT * FROM mk;
```

```
##   id_mhs    nama     jurusan tanggal_lahir
## 1      1    Agus   Manajemen          <NA>
## 2      2    Budi   Akuntansi          <NA>
## 3      3 Chintia Informatika          <NA>
## 4      4    Dewi   Akuntansi          <NA>

##   kode_mk              nama_mk semester
## 1       1     Matematika Dasar       NA
## 2       2 Pengantar Data Sains       NA
## 3       3                Agama       NA
```

Selanjutnya kita akan memperbaharui data `tanggal lahir` mahasiswa, dan data `semester` setiap mata kuliah. Perintah `UPDATE` dapat digunakan untuk mengisi nilai ke kolom tersebut berdasarkan ID tertentu. Misalkan, kita akan menambahkan `tanggal lahir` mahasiswa dengan nama **Agus** yang lahir pada tanggal 10 Oktober 2000 (Ingat untuk menggunakan `ID` sebagai **primary key**, *jangan beracuan pada nama*).

```
UPDATE mahasiswa
SET tanggal_lahir = '2000-10-10' -- Format Penulisan YYYY-MM-DD
WHERE id_mhs = 1; -- Merupakan ID dari Agus
```

Cara ini dapat dilakukan untuk beberapa ID dengan menggunakan beberapa pernyataan UPDATE. Agar lebih efisien, dapat digunakan satu pernyataan untuk mengupdate beberapa baris jika ingin memberikan nilai yang berbeda berdasarkan ID. Berikut adalah contoh untuk mengupdate beberapa ID sekaligus:

```
UPDATE mahasiswa
SET tanggal_lahir = CASE
            WHEN id_mhs = 2 THEN '2001-01-11'
            WHEN id_mhs = 3 THEN '2002-12-31'
            WHEN id_mhs = 4 THEN '2001-10-08'
            ELSE tanggal_lahir  -- Jika ID tid_mhsak cocok, tetap gunakan nilai yang ada
          END
WHERE id_mhs IN (2, 3, 4);
```

Selanjutnya, kita dapat melihat tabel `mahasiswa` yang sudah terdapat informasi `tanggal_lahir`.

```
SELECT * FROM mahasiswa;
```

```
##   id_mhs    nama     jurusan tanggal_lahir
## 1      1    Agus   Manajemen    2000-10-10
## 2      2    Budi   Akuntansi    2001-01-11
## 3      3 Chintia Informatika    2002-12-31
## 4      4    Dewi   Akuntansi    2001-10-08
```

1.2. Latihan

1. Diketahui mata kuliah Agama, Matematika Dasar, dan Pengantar Data Sains masing-masing diampu di semester 1,2, dan 5. Perbaharui informasi ini pada tabel `mk`. Tuliskan bagaimana perintah SQLnya.
2. Masukkan mata kuliah berikut ke dalam tabel `mk`: Fisika Dasar (semester 1); Kalkulus 3 (semester 3); Biologi (semester 4). Tuliskan bagaimana perintah SQLnya.
3. Tambahkan kolom `hari_kuliah` dengan tipe data VARCHAR(6). Tuliskan bagaimana perintah SQLnya.
4. Masukkan informasi hari dari mata kuliah berikut:

   Matematika Dasar: Senin
   Pengantar Data Sains: Rabu
   Agama: Jumat
   Fisika Dasar: Selasa
   Kalkulus 3: Kamis
   Biologi: Rabu

5. Tunjukkan hasil dari tabel `mk` yang terbaru.

2. Mengurutkan Data

Pengurutan Data adalah salah satu fungsi penting dalam SQL yang digunakan untuk menampilkan data dalam urutan tertentu. Kita bisa mengurutkan data berdasarkan satu atau lebih kolom, baik secara **ascending** (menaik) maupun **descending** (menurun).

Struktur perintah SQL untuk mengurutkan data adalah sebagai berikut.

```
SELECT kolom1, kolom2, ...
FROM nama_tabel
ORDER BY kolom ASC|DESC;
```

- **ORDER BY**: Menentukan kolom yang menjadi dasar pengurutan.
- **ASC**: Untuk pengurutan menaik (*ascending*). Ini adalah pengurutan **default**, dapat diabaikan jika ingin menaik.
- **DESC**: Untuk pengurutan menurun (*descending*).

2.1. Pengurutan berdasarkan satu kolom

Dari tabel `mahasiswa` sebelumnya, misalkan kita ingin mengurutkan berdasarkan **jurusan** dalam urutan *menaik*, maka kita dapat menggunakan perintah berikut.

```
SELECT * FROM mahasiswa
ORDER BY jurusan ASC;
```

```
##   id_mhs    nama     jurusan tanggal_lahir
## 1      2    Budi   Akuntansi    2001-01-11
## 2      4    Dewi   Akuntansi    2001-10-08
## 3      3 Chintia Informatika    2002-12-31
## 4      1    Agus   Manajemen    2000-10-10
```

2.2. Pengurutan berdasarkan beberapa kolom

Kita juga bisa mengurutkan berdasarkan lebih dari satu kolom. Misalnya, mengurutkan tabel `mahasiswa` berdasarkan **jurusan** dalam urutan *menaik*, lalu **tanggal_lahir** dalam urutan *menurun*.

```
SELECT * FROM mahasiswa
ORDER BY jurusan ASC, tanggal_lahir DESC;
```

```
##   id_mhs    nama     jurusan tanggal_lahir
## 1      4    Dewi   Akuntansi    2001-10-08
## 2      2    Budi   Akuntansi    2001-01-11
## 3      3 Chintia Informatika    2002-12-31
## 4      1    Agus   Manajemen    2000-10-10
```

2.3. Latihan Soal

1. Urutkan tabel `mk` berdasarkan **nama_mk** pada urutan *menurun*. Tuliskan bagaimana perintah SQLnya dan tunjukkan hasil tabel yang sudah diurutkan.
2. Urutkan tabel `mk` berdasarkan **hari_kuliah** dalam urutan *menurun*, lalu **semester** dalam urutan *menaik*. Tuliskan bagaimana perintah SQLnya dan tunjukkan hasil tabel yang sudah diurutkan.

3. Menggabungkan Tabel

Dalam basis data relasional, tabel-tabel sering kali memiliki hubungan satu sama lain. **Penggabungan data** adalah proses untuk mengambil data dari beberapa tabel dan menampilkannya bersama-sama dalam hasil query.

`JOIN` adalah perintah dalam SQL yang digunakan untuk menggabungkan baris dari dua atau lebih tabel berdasarkan kolom terkait antara tabel-tabel tersebut. Fungsinya adalah untuk mengumpulkan data dari beberapa tabel yang memiliki hubungan, biasanya dengan menggunakan kolom yang sama atau berelasi, seperti **primary key** dan **foreign key**.

Struktur perintah SQL untuk menggabungkan data adalah sebagai berikut.

```sql
SELECT kolom1, kolom2, ...
FROM tabel1
JOIN jenis_join tabel2
ON tabel1.kolom = tabel2.kolom;
```

- **JOIN**: Kata kunci untuk menggabungkan tabel.
- **ON**: Menentukan kondisi penggabungan, yaitu kolom yang menghubungkan kedua tabel.

Ada beberapa jenis join dalam SQL, diantaranya yaitu: `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, dan `FULL OUTER JOIN`.

3.1. INNER JOIN

`INNER JOIN` adalah jenis join dalam SQL yang digunakan untuk menggabungkan data dari dua tabel berdasarkan kecocokan di antara kolom yang terkait. `INNER JOIN` hanya akan menampilkan baris-baris yang memiliki kecocokan di kedua tabel; baris yang tidak memiliki pasangan di tabel lain tidak akan ditampilkan.

Contoh Penggunaan `INNER JOIN`

Misalkan kita memiliki dua tabel: `karyawan` dan `proyek`

```sql
-- Buat tabel karyawan
CREATE TABLE karyawan (
    id_karyawan INT AUTO_INCREMENT PRIMARY KEY,
    nama_karyawan VARCHAR(100),
    jabatan VARCHAR(100)
);
INSERT INTO karyawan
VALUES
(NULL, 'Andi', 'Insinyur'),
(NULL, 'Budi', 'Teknisi'),
(NULL, 'Citra', 'K3'),
(NULL, 'Dodi', 'Insinyur');

-- Buat tabel proyek
CREATE TABLE proyek (
    id_proyek INT PRIMARY KEY,
    nama_proyek VARCHAR(100),
```

```
    id_karyawan INT
);
INSERT INTO proyek
VALUES
(101, 'Jalan', '1'),
(102, 'Hotel', '2'),
(103, 'Jembatan', '1'),
(104, 'Kantor', '3'),
(105, 'Waduk', NULL);

SELECT * FROM karyawan;
SELECT * FROM proyek;
```

```
##   id_karyawan nama_karyawan  jabatan
## 1           1          Andi Insinyur
## 2           2          Budi  Teknisi
## 3           3         Citra       K3
## 4           4          Dodi Insinyur
```

```
##   id_proyek nama_proyek id_karyawan
## 1       101       Jalan           1
## 2       102       Hotel           2
## 3       103    Jembatan           1
## 4       104      Kantor           3
## 5       105       Waduk          NA
```

Kita ingin mendapatkan daftar karyawan yang terlibat dalam proyek, beserta nama proyek yang mereka kerjakan. Kita dapat menggunakan `INNER JOIN` untuk menggabungkan tabel **karyawan** dan **proyek** berdasarkan `id_karyawan`.

```
SELECT karyawan.nama_karyawan, karyawan.jabatan, proyek.nama_proyek
FROM karyawan
INNER JOIN proyek
ON karyawan.id_karyawan = proyek.id_karyawan;
```

```
##   nama_karyawan  jabatan nama_proyek
## 1          Andi Insinyur       Jalan
## 2          Budi  Teknisi       Hotel
## 3          Andi Insinyur    Jembatan
## 4         Citra       K3      Kantor
```

Hanya karyawan yang terlibat dalam proyek yang akan ditampilkan. Dalam contoh ini, karyawan **Dodi** tidak muncul dalam hasil karena dia tidak terlibat dalam proyek mana pun (kolom `id_karyawan` di tabel proyek bernilai `NULL`).

3.2. LEFT JOIN

`LEFT JOIN` (atau `LEFT OUTER JOIN`) adalah jenis join dalam SQL yang digunakan untuk menggabungkan data dari dua tabel. `LEFT JOIN` akan mengembalikan semua baris dari tabel kiri (tabel pertama) dan hanya baris yang cocok dari tabel kanan (tabel kedua). Jika tidak ada kecocokan, kolom dari tabel kanan akan diisi dengan nilai `NULL`.

Contoh Penggunaan `LEFT JOIN`

Dari tabel `karyawan` dan `proyek` pada kasus sebelumnya, ingin didapatkan **daftar seluruh karyawan** beserta proyek yang mereka kerjakan. Jika seorang karyawan tidak terlibat dalam proyek mana pun, kita tetap ingin melihat nama karyawan tersebut dengan kolom proyek yang bernilai `NULL`.

```
SELECT karyawan.nama_karyawan, karyawan.jabatan, proyek.nama_proyek
FROM karyawan
LEFT JOIN proyek
ON karyawan.id_karyawan = proyek.id_karyawan;
```

```
##   nama_karyawan  jabatan nama_proyek
## 1          Andi Insinyur    Jembatan
## 2          Andi Insinyur       Jalan
## 3          Budi  Teknisi       Hotel
## 4         Citra       K3      Kantor
## 5          Dodi Insinyur        <NA>
```

Terlihat bahwa **Andi**, **Budi**, dan **Citra** muncul dengan proyek yang mereka kerjakan. **Dodi** juga muncul dalam hasil, tetapi kolom `nama_proyek` bernilai `NULL` karena **Dodi** tidak terlibat dalam proyek mana pun. `LEFT JOIN` memastikan bahwa semua data dari tabel kiri (**karyawan**) ditampilkan, terlepas dari apakah mereka memiliki pasangan di tabel kanan (**proyek**).

3.3. RIGHT JOIN

`RIGHT JOIN` (atau `RIGHT OUTER JOIN`) adalah jenis join dalam SQL yang mengembalikan semua baris dari tabel kanan (tabel kedua) dan hanya baris yang cocok dari tabel kiri (tabel pertama). Jika tidak ada kecocokan, kolom dari tabel kiri akan diisi dengan nilai `NULL`.

Contoh Penggunaan `RIGHT JOIN`

Dari tabel `karyawan` dan `proyek` pada kasus sebelumnya, ingin didapatkan **daftar seluruh proyek** dan karyawan yang mengerjakannya. Jika sebuah proyek tidak memiliki karyawan yang terlibat, kita tetap ingin melihat proyek tersebut dengan kolom karyawan bernilai `NULL`.

```
SELECT karyawan.nama_karyawan, karyawan.jabatan, proyek.nama_proyek
FROM karyawan
RIGHT JOIN proyek
ON karyawan.id_karyawan = proyek.id_karyawan;
```

```
##   nama_karyawan  jabatan nama_proyek
## 1          Andi Insinyur       Jalan
## 2          Budi  Teknisi       Hotel
## 3          Andi Insinyur    Jembatan
## 4         Citra       K3      Kantor
## 5          <NA>     <NA>       Waduk
```

Terlihat bahwa Proyek Jalan, Hotel, Jembatan, dan Kantor muncul dengan karyawan yang terlibat. Selain itu, Proyek Waduk juga muncul dalam hasil, tetapi kolom `nama_karyawan` dan `jabatan` bernilai `NULL` karena tidak ada karyawan yang terlibat dalam proyek tersebut. `RIGHT JOIN` memastikan bahwa semua data dari tabel kanan (**proyek**) ditampilkan, terlepas dari apakah mereka memiliki pasangan di tabel kiri (**karyawan**).

3.4. FULL OUTER JOIN

`FULL OUTER JOIN` adalah jenis join dalam SQL yang mengembalikan semua baris dari kedua tabel, baik yang memiliki kecocokan maupun yang tidak. Jika suatu baris dari tabel kiri tidak memiliki pasangan di tabel kanan, kolom dari tabel kanan akan berisi `NULL`, dan sebaliknya. `FULL OUTER JOIN` sangat bermanfaat saat kita ingin melihat gambaran lengkap dari kedua tabel tanpa kehilangan informasi dari salah satu tabel.

MySQL tidak mendukung `FULL OUTER JOIN` secara langsung. Namun, kita bisa membuatnya dengan menggabungkan hasil dari `LEFT JOIN` dan `RIGHT JOIN` menggunakan `UNION` untuk mendapatkan hasil yang serupa:

Contoh Penggunaan `FULL OUTER JOIN` di MySQL menggunakan `UNION`

Dari tabel `karyawan` dan `proyek` pada kasus sebelumnya, ingin didapatkan daftar semua karyawan beserta proyek yang mereka kerjakan, termasuk proyek yang tidak memiliki karyawan yang terlibat.

```
SELECT karyawan.nama_karyawan, karyawan.jabatan, proyek.nama_proyek
FROM karyawan
LEFT JOIN proyek
ON karyawan.id_karyawan = proyek.id_karyawan
UNION
SELECT karyawan.nama_karyawan, karyawan.jabatan, proyek.nama_proyek
FROM karyawan
RIGHT JOIN proyek
ON karyawan.id_karyawan = proyek.id_karyawan;
```

```
##   nama_karyawan  jabatan nama_proyek
## 1          Andi Insinyur    Jembatan
## 2          Andi Insinyur       Jalan
## 3          Budi  Teknisi       Hotel
## 4         Citra       K3      Kantor
## 5          Dodi Insinyur        <NA>
## 6          <NA>     <NA>       Waduk
```

LEFT JOIN menggabungkan data dari tabel `karyawan` dengan `proyek`, menampilkan semua baris dari tabel `karyawan`, termasuk yang tidak memiliki kecocokan dengan `proyek`. **RIGHT JOIN** menggabungkan data dari tabel `proyek` dengan `karyawan`, menampilkan semua baris dari tabel `proyek`, termasuk yang tidak memiliki kecocokan dengan `karyawan`. **UNION** menggabungkan hasil dari `LEFT JOIN` dan `RIGHT JOIN` untuk menghasilkan efek seperti `FULL OUTER JOIN`.

Dari hasil yang ditampilkan, terlihat bahwa baris yang memiliki kecocokan di kedua tabel muncul dengan lengkap, seperti karyawan yang bekerja pada proyek tertentu. Baris dari tabel karyawan yang tidak memiliki pasangan di tabel proyek (seperti **Dodi**) tetap muncul, tetapi kolom proyek bernilai `NULL`. Baris dari tabel proyek yang tidak memiliki pasangan di tabel karyawan (seperti **Proyek Waduk**) juga muncul, tetapi kolom karyawan bernilai `NULL`. Dengan solusi ini, akan didapatkan seluruh baris dari kedua tabel, dengan data yang tidak cocok diisi `NULL` pada kolom yang tidak sesuai.

3.5. Latihan Soal

Anggaplah Anda memiliki dua tabel berikut:

Tabel `siswa`

id_siswa	nama_siswa	kelas
1	Ali	10
2	Budi	11
3	Citra	10
4	Dani	12
5	Elsa	11

Tabel `nilai`

id_nilai	id_siswa	mata_pelajaran	nilai
1	1	Matematika	85
2	2	Biologi	90
3	3	Fisika	88
4	1	Kimia	78
5	6	Matematika	92

Dari masing-masing soal-soal berikut, tuliskan bagaimana perintah SQLnya dan tunjukkan hasil tabel/luarannya.

1. Tampilkan semua siswa yang memiliki nilai di mata pelajaran tertentu, beserta nilai yang mereka peroleh.
2. Siapa saja siswa yang memiliki nilai untuk mata pelajaran "Matematika"?
3. Berapa jumlah siswa yang memiliki nilai dalam mata pelajaran "Fisika"?
4. Tampilkan semua siswa, beserta nilai mereka (jika ada). Jika seorang siswa belum memiliki nilai, tampilkan `NULL` di kolom nilai.
5. Tampilkan daftar siswa yang tidak memiliki nilai di mata pelajaran apa pun.
6. Cari siswa di kelas 11 yang memiliki nilai di mata pelajaran "Biologi" atau yang belum memiliki nilai sama sekali.
7. Tampilkan semua nilai yang tercatat, beserta nama siswa yang mendapatkannya. Jika tidak ada siswa yang cocok, tampilkan `NULL` di kolom nama siswa.
8. Apakah ada nilai yang tercatat untuk id_siswa yang tidak ada di tabel `siswa`?
9. Tampilkan semua siswa beserta nilai "Matematika" mereka (termasuk siswa yang tidak memiliki nilai Matematika).
10. Tampilkan semua siswa dan semua nilai yang tercatat, termasuk yang tidak memiliki pasangan. Jika siswa tidak memiliki nilai atau jika nilai tidak memiliki siswa terkait, kolom akan berisi `NULL`.
11. Tampilkan semua siswa, termasuk yang tidak memiliki nilai, dan semua nilai, termasuk yang tidak memiliki siswa terkait.
12. Tampilkan daftar siswa kelas 10 yang memiliki atau tidak memiliki nilai di mata pelajaran apa pun.